AI-Driven Edge–Cloud Cyclone Monitoring

What the system does, how the files connect, and what the dashboard shows. A 10-minute read.

1. The big picture in six steps

We are simulating a **cyclone monitoring network**. Data travels from the sea to a screen like this:

1. **Sensors** (buoys and radars at sea) measure wind, temperature and pressure every second and send each reading as a small packet.
2. A **satellite station** takes big cloud images. Instead of sending the whole 2 MB image, it analyses it on the spot and sends a ~1 KB summary, e.g. “there's a spiral storm, risk 92”.
3. All packets go to the **edge gateway**, which sorts them by importance and sends them over a slow link to the server, **most important first**.
4. Some packets get **lost** on the way, as real radio links lose data in bad weather. The **server** notices which ones are missing and estimates what they probably said, using a small neural network (LSTM).
5. The server calculates a **cyclone risk score from 0 to 100**. Above 60 it declares an **alert** and tells the gateway, which tells the sensors. On real hardware an LED and buzzer switch on.
6. The **dashboard** shows all of this live in the browser.

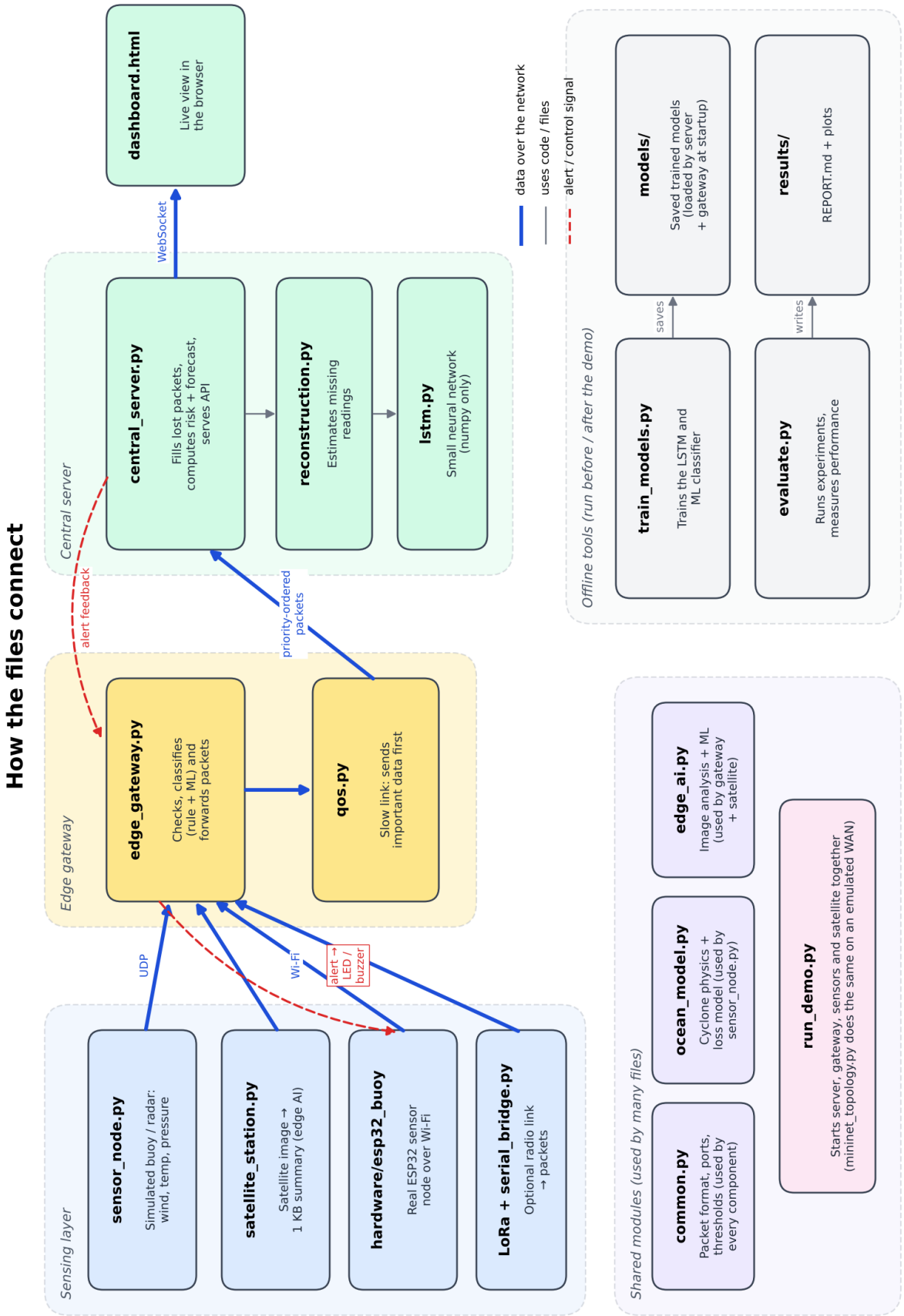
In the demo, a fake cyclone starts about 15 seconds in and moves towards the sensors, so you watch the system go from calm to alert and back.

Three ideas to remember

Idea	In one sentence
Edge AI	Process big data (images) near where it's collected and send only the useful summary.
QoS (Quality of Service)	When bandwidth is short, critical cyclone data goes first and unimportant data waits or is dropped.
Packet-loss recovery	When packets go missing, estimate them with an LSTM instead of leaving gaps, and replace the estimate if the real packet arrives late.

2. How the files connect

Turn the page sideways to read the diagram.

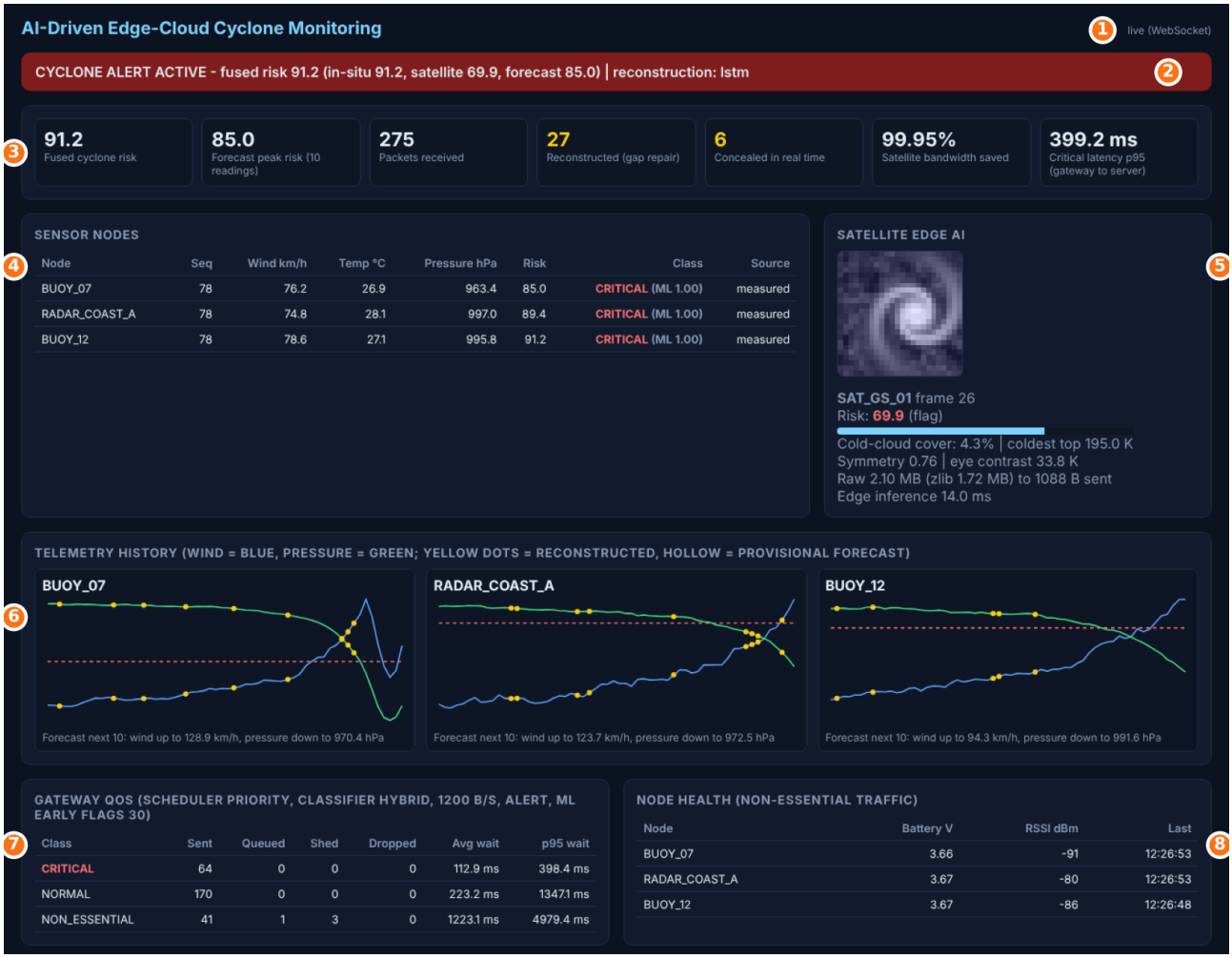


Read it left to right. Data starts in the sensing layer, passes through the edge gateway, reaches the central server and ends on the dashboard. The red dashed arrows show the alert travelling back. The bottom row holds code shared by many files and the offline tools for training and testing.

File	What it does, in one line
sensor_node.py	Pretends to be a buoy/radar and sends readings; deliberately drops some packets.
satellite_station.py	Makes satellite images, analyses them, sends only a small summary.
edge_gateway.py	Receives everything, checks it, labels it CRITICAL / NORMAL / NON-ESSENTIAL.
qos.py	Decides the sending order on the slow link: important data first.
central_server.py	Detects lost packets, fills them in, computes risk, runs the website.
reconstruction.py + lstm.py	The neural network that predicts missing readings.
edge_ai.py	Image analysis for the satellite, plus the ML early-warning classifier.
common.py / ocean_model.py	Shared packet format and thresholds / the fake cyclone physics.
run_demo.py	Starts everything with one command.
train_models.py / evaluate.py	Trains the models / measures how well everything works.

3. The dashboard, piece by piece

Open http://127.0.0.1:8000 after running `python run_demo.py`. The numbers on the picture match the sections below.



1 “live (WebSocket)” (top-right)

The page is connected to the server and updates every second by itself. If it says “polling”, it is still working, just through a slightly slower method.

2 The banner (green or red)

Example: *“CYCLONE ALERT ACTIVE – fused risk 91.2 (in-situ 91.2, satellite 69.9, forecast 85.0)”*

- **Green** = normal (risk below 60). **Red** = alert active.
- **Fused risk** is the final score, combining three opinions: **in-situ** (from the actual sensor readings; “in-situ” means measured on the spot), **satellite** (from the cloud images) and **forecast** (what the LSTM predicts for the next 10 readings). It is mainly the highest of the three, so any one of them can raise the alarm.
- The alert turns **on at 60** but only turns **off below 50**, so it doesn't flicker when the risk hovers around 60 (this is called hysteresis).
- **reconstruction: lstm** means lost packets are being filled in by the neural network.

3 The seven number boxes

Box	Meaning
Fused cyclone risk	Same as the banner, 0–100.
Forecast peak risk	The worst risk the LSTM predicts over the next 10 readings: where things are heading.
Packets received	Total packets that reached the server.
Reconstructed (gap repair)	Lost packets filled in *after* the next one arrived. If 10 and 13 arrived but 11 and 12 didn't, the server estimates 11 and 12.
Concealed in real time	Guesses made *while still waiting*: if a sensor goes quiet too long, a temporary prediction is shown, then replaced when real data arrives.
Satellite bandwidth saved	99.95% means about 1 KB was sent instead of a 2 MB image.
Critical latency p95	Time for critical packets from gateway to server. “p95” means 95% arrived faster than this (e.g. 399 ms).

4 Sensor nodes table

One row per sensor, showing its **latest** reading.

Column	Meaning
Seq	Sequence number. Every packet is numbered 1, 2, 3… A jump (10 → 13) tells the server packets were lost.
Wind / Temp / Pressure	The reading. A cyclone means **high wind and low pressure** (normal ≈ 1010 hPa; below 990 is dangerous).
Risk	Risk score for this one sensor.
Class	The gateway's label: **CRITICAL** (red) or NORMAL. “(ML 1.00)” is the ML classifier's confidence that a cyclone is coming (0 = calm, 1 = certain).
Source	**measured** = real packet; **reconstructed** = estimate for a lost packet; **forecast** = temporary real-time guess; **late packet** = arrived late and replaced its estimate.

*A packet can be CRITICAL while the wind is still below 60. That is the **ML early warning**: it saw the pressure dropping fast and flagged danger before the thresholds were crossed.*

5 Satellite edge AI panel

- **Spiral picture:** a tiny 24×24 thumbnail of the cloud image (bright = cold, high storm clouds).
- **Risk / flag:** the satellite's risk score; “flag” appears at 60 or above.
- **Cold-cloud cover:** percentage of the image with cloud tops colder than 220 K (about −53 °C), the sign of powerful storm clouds. **Coldest top** is the coldest cloud temperature.
- **Symmetry** (0–1): how round and organised the storm is. **Eye contrast:** how much warmer the centre is than the ring around it; a clear warm eye means a mature cyclone.
- **Raw 2.10 MB (zlib 1.72 MB) → 1088 B sent:** normal compression only reaches 1.72 MB, while our edge AI sends about 1 KB. **Edge inference** is the time taken to analyse the image (≈14 ms).

6 Telemetry history charts

- One chart per sensor, covering the last 60 readings. **Blue** = wind, **green** = pressure, **red dashed line** = 60 km/h danger threshold.
- **Yellow filled dots** = reconstructed values (lost packets estimated by the LSTM). **Hollow yellow dots** = temporary real-time guesses.
- As the cyclone arrives, blue rises above the red line and green drops; when it passes, both recover.
- “**Forecast next 10…**” shows the LSTM's predicted peak wind and lowest pressure coming up.

7 Gateway QoS table: “important data first”

The heading, e.g. *“scheduler priority, classifier hybrid, 1200 B/s, ALERT, ML early flags 30”*, means: QoS is on (**fifo** would mean first come, first served); a packet is critical if the threshold rule *or* the ML says so; the link is limited to 1200 bytes/second; the gateway is in alert mode; and the ML marked 30 packets critical before the rule would have.

Column	Meaning
Sent	Packets forwarded to the server.
Queued	Packets waiting right now.
Shed	Unimportant packets deliberately dropped because the link was congested (only NON_ESSENTIAL).
Dropped	Packets lost because the queue was completely full (ideally 0).
Avg / p95 wait	How long packets waited at the gateway.

Compare the rows: CRITICAL waits the least and NON_ESSENTIAL the most. That is prioritisation working.

8 Node health table

The NON_ESSENTIAL “housekeeping” packets each sensor sends every 5 readings: **Battery V** (battery voltage), **RSSI dBm** (radio signal strength; −60 is strong, −95 is weak) and **Last** (time of the last health packet). This is the data the gateway drops first when bandwidth is tight. If it stops updating during an alert, shedding is doing its job.

4. What to watch during one demo run

Time	What happens on the dashboard
0–15 s	Green banner, low risk, everything NORMAL, flat lines.
~15–30 s	Pressure starts dropping; ML confidence rises and some packets turn CRITICAL early.
~30–50 s	Wind crosses the red line, the banner turns red, the satellite spiral brightens and shows an eye.
Whole run	Yellow dots appear wherever packets were lost.
During alert	In the QoS table CRITICAL stays fast while NORMAL / NON_ESSENTIAL wait longer; “Shed” may rise.
End	The storm passes, wind falls, pressure recovers; the alert clears when risk drops below 50.

5. Key results to quote

What we measured	Result
Satellite bandwidth	2.1 MB image → ~1 KB summary: **99.95% less data**.
Critical data on a congested link (1200 B/s)	95% arrive within **0.36 s with QoS**, versus **49 s** without.
Very tight link (1000 B/s)	Without QoS **23% of critical packets are lost**; with QoS, none.
ML early warning	Flags danger about **16 readings before** the thresholds are crossed (cost: some false alarms).
Real-time guessing of missing data	LSTM error ~27% lower for wind and ~55% lower for pressure than repeating the last value.

These numbers come from simulated data (results/REPORT.md). Testing on real buoy data is the next step.

6. Questions your teacher might ask

Q: Why UDP and not TCP?

A: Marine and satellite links lose packets. TCP would keep re-sending them and add delay; with UDP we see the loss and handle it ourselves by prioritising and reconstructing.

Q: Why process satellite images at the edge?

A: Sending raw images would fill the slow link. Sending a 1 KB summary leaves room for the critical sensor data.

Q: What does the LSTM actually do?

A: It learns how wind, temperature and pressure change over time. Given the last 12 readings, it predicts the next one. We use that to fill gaps and to guess overdue packets.

Q: How is QoS different from just a priority queue?

A: During an alert we keep a small reserve of bandwidth free so a critical packet never waits behind a normal one, and we drop non-essential data only when the link is actually congested, so no capacity is wasted.

Q: What's the role of the hardware?

A: An ESP32 with a pressure sensor replaces a simulated buoy. It sends the same packets, and its LED/buzzer turn on when the system declares an alert.